CAS 760 — Development 2 Project: A Simple File System in Alonzo

Suleyman Kiani Hamza Jamil

April 16, 2026

Contents

1 Introduction **2**

2 Permissions **3**

3 Files **4**

4 Secured Files **5**

5 Directories **6**

6 Directory Development **8**

7 Theory Morphism: Files to Directories **9**

8 Examples **11**

9 Conclusion **12**

A Proofs and Validations for DIR-DEV **13**

A.1 Validation of Def 1 (contains) . . . 13
A.2 Thm 1: TOTAL2(contains) . . . 13
A.3 Thm 2: $\forall\, n : N$. contains n emptydir $= F_o$ . . . 13
A.4 Validation of Def 2 (size) . . . 13
A.5 Thm 3: TOTAL(size) . . . 14
A.6 Thm 4: size emptydir $= 0$ . . . 14
A.7 Thm 5: $\forall\, f : F,\, d : D$. size (addfile f d) $=$ size $d + 1$ . . . 14
A.8 Thm 6: $\forall\, d : D$. size $d \geq 0$ . . . 14

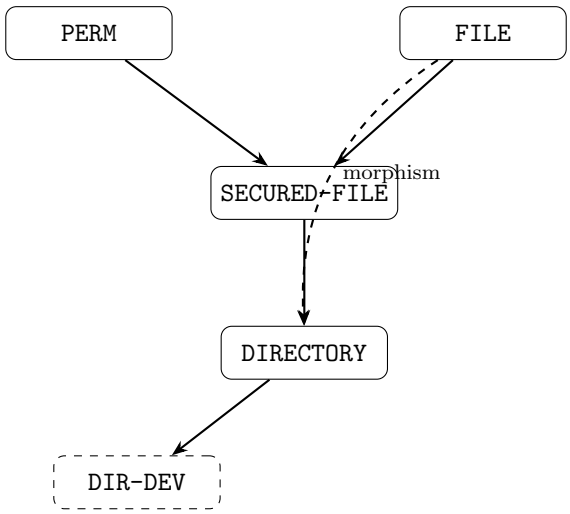
1 Introduction

This paper formalizes a simple file system as a development graph in Alonzo, following the little theories method [1]. File systems are a fundamental concept in computing: they organize data into named, content-bearing objects (files), assign permissions to control access, and group files into directories. Each of these concepts corresponds naturally to a theory in the little theories framework.

We build a hierarchy of increasingly complex structures. The base theories PERM and FILE capture permissions and basic files independently. The theory extension SECURED-FILE combines both, associating a permission level with each file. A further extension DIRECTORY introduces directories as inductive structures over secured files. Key operations (lookup, containment, directory size) are developed in DIR-DEV, and a theory morphism shows that single files embed into the directory structure.

The development follows the module format established in the textbook and the monoids paper of Farmer & Zvigelsky [2]. We use compact notation throughout, dropping type subscripts where unambiguous, consistent with Notational Convention 5 of [1].

Theory graph.



Solid boxes are theories (theory definitions or extensions). The dashed box is a development definition. Solid arrows denote theory extensions. The dashed arrow denotes the theory morphism FILE-to-DIR.

Work division.

- **Suleyman Kiani**: FILE, SECURED-FILE, DIRECTORY, DIR-DEV (Sections 3–6), including all proofs and validations.
- **Hamza Jamil**: PERM, theory morphism FILE-to-DIR (Sections 2 and 7), examples, and presentation lead.

2 Permissions

A file system associates *permissions* with files to control access. We model a simple permission hierarchy with four levels: no access, read-only, read-write, and administrator. These form an enumeration inductive type — a finite set freely generated by four nullary constructors, analogous to the theory of Boolean values (BOOLE) in Chapter 10 of [1].

Theory Definition 2.1 (Permissions)**.**

Name: PERM.

Base types: P.

Constants: $\mathsf{noaccess}_P$, rd_P, wr_P, admin_P.

Axioms:

1. $\mathsf{DISTINCT}(\mathsf{noaccess}, \mathsf{rd}, \mathsf{wr}, \mathsf{admin})$ (all constructors are distinct).
2. $\forall\, p : \{P\} \,.\, p\ \mathsf{noaccess} \ \wedge\ p\ \mathsf{rd} \ \wedge\ p\ \mathsf{wr} \ \wedge\ p\ \mathsf{admin} \Rightarrow \forall\, x : P \,.\, p\ x$ (structural induction).

Explanation. Axiom 1 states that the four permission values are pairwise distinct (using the DISTINCT abbreviation from Table 6.2 of [1]). Axiom 2 is the structural induction principle: any predicate that holds for all four constructors holds for every element of P. Together, these axioms make (P, K_P) an enumeration inductive type where $K_P = \{\mathsf{noaccess}, \mathsf{rd}, \mathsf{wr}, \mathsf{admin}\}$, so in every standard model, P has exactly four elements. (We write K_P for the set of constructors of P, following the standard inductive-type convention; the subscript also keeps the notation distinct from the base type C of FILE.)

3 Files

A *file* is a named container for content. In a real file system, a file has (at minimum) a name and content. We model this as a theory with a constructor mkfile that builds a file from a name and content, and accessor functions fname and fcontent that extract the components.

This theory defines the space of all possible files over arbitrary name and content types. It is analogous to a simple record type `type file = {name : N; content : C}` in OCaml.

Theory Definition 3.1 (Files)**.**

Name: FILE.

Base types: N, C, F.

Constants: $\mathsf{mkfile}_{N \to C \to F}$, $\mathsf{fname}_{F \to N}$, $\mathsf{fcontent}_{F \to C}$.

Axioms:

1. $\mathsf{TOTAL2}(\mathsf{mkfile})$ (mkfile is total).
2. $\mathsf{INJ2}(\mathsf{mkfile})$ (mkfile is injective).
3. $\forall\, n : N,\, c : C \,.\, \mathsf{fname}\ (\mathsf{mkfile}\ n\ c) = n$ (name accessor).
4. $\forall\, n : N,\, c : C \,.\, \mathsf{fcontent}\ (\mathsf{mkfile}\ n\ c) = c$ (content accessor).
5. $\mathsf{TOTAL}(\mathsf{fname})$ (fname is total).
6. $\mathsf{TOTAL}(\mathsf{fcontent})$ (fcontent is total).
7. $\forall\, f : F \,.\, \mathsf{mkfile}\ (\mathsf{fname}\ f)\ (\mathsf{fcontent}\ f) = f$ (surjectivity of mkfile).

Explanation of the axioms. Axiom 1 states that any name–content pair produces a valid file. Axiom 2 ensures distinct pairs produce distinct files ("no confusion"). Axioms 3–4 define the accessor functions via their interaction with the constructor. Axioms 5–6 ensure the accessors are total on all files. Axiom 7 establishes that every file is constructible — i.e., mkfile is surjective, so the type F is exactly the set of name–content pairs. Together, Axioms 1–2 and 7 make F isomorphic to $N \times C$.

Design note. We could alternatively model files using the product type $N \times C$ with the built-in projection functions fst and snd. We choose an abstract type F with explicit accessors to allow extension with additional attributes (e.g., permissions) via theory extensions, which is more natural in the little theories setting.

4 Secured Files

A *secured file* is a file that has been assigned a permission level. Conceptually, SECURED-FILE combines FILE and PERM: it associates a permission from PERM with each file from FILE. We present SECURED-FILE as a joint extension of both parent theories, so that the base type P is inherited from PERM (not reintroduced), and the base types N, C, F and the constants mkfile, fname, fcontent are inherited from FILE. This is the standard pushout-style construction for combining two theories that share no non-trivial sub-theory, following the little theories method of [1]: the two extension arrows FILE $\to$ SECURED-FILE and PERM $\to$ SECURED-FILE in the theory graph of Section 1 encode the import of each parent's language into SECURED-FILE. SECURED-FILE then introduces only the new constants perm and protect, together with axioms relating them to the inherited constants.

Theory Extension 4.1 (Secured Files)**.**

Name: SECURED-FILE.

Extends FILE and PERM.

New base types: None (P is inherited from PERM).

New constants: $\mathsf{perm}_{F\to P}$, $\mathsf{protect}_{F\to P\to F}$.

New axioms:

8. $\mathsf{TOTAL}(\mathsf{perm})$ (perm is total).
9. $\mathsf{TOTAL2}(\mathsf{protect})$ (protect is total).
10. $\forall\, f : F,\, p : P \,.\, \mathsf{perm}\ (\mathsf{protect}\ f\ p) = p$ (protect sets permission).
11. $\forall\, f : F,\, p : P \,.\, \mathsf{fname}\ (\mathsf{protect}\ f\ p) = \mathsf{fname}\ f$ (protect preserves name).
12. $\forall\, f : F,\, p : P \,.\, \mathsf{fcontent}\ (\mathsf{protect}\ f\ p) = \mathsf{fcontent}\ f$ (protect preserves content).

Explanation of the axioms. Axiom 8 states that every file has an assigned permission. Axiom 9 ensures that applying a permission always succeeds. Axiom 10 defines the core semantics of protect: after applying protect f p, the permission of the resulting file is p. Axioms 11–12 guarantee that protect only changes the permission — it does not alter the file's name or content.

SECURED-FILE inherits the types and constants of both FILE and PERM (including the base type P of permission levels) and adds two new constants, perm and protect. The axiom numbering continues from FILE (Axioms 1–7), consistent with the convention for theory extensions (Section 9.2 of [1]). The language of SECURED-FILE is the union of the languages of FILE and PERM, augmented with the constants perm and protect.

5 Directories

A *directory* is a collection of secured files organized as a flat structure. We model this as an inductive type: a directory is either empty or formed by adding a secured file to an existing directory. This mirrors the standard inductive structure of lists (cf. the parallel between PA and lists in Theory Definition 9.7 of [1]).

We add two new constructors: emptydir (the empty directory, analogous to nil in lists) and addfile (add a file to a directory, analogous to cons).

Design note on flat structure. The constructor addfile has type $F \to D \to D$: it takes a *file* (not a directory) as its first argument. This makes DIRECTORY a flat container of SECURED-FILE entries, mirroring the LIST pattern in the textbook. A hierarchical model would require a constructor $\mathsf{adddir} : D \to D \to D$ allowing directories to contain directories, yielding a tree-shaped inductive type. The flat choice is intentional: it keeps the inductive type within the standard fourth-kind constructor pattern of Chapter 10 of [1], preserves the direct analogy with lists, and permits the clean FILE-to-DIR morphism in Section 7, which relies on unique extraction from single-entry directories. A hierarchical extension is a natural direction for future work.

Theory Extension 5.1 (Directories).

Name: DIRECTORY.

Extends SECURED-FILE.

New base types: D.

New constants: $\mathsf{emptydir}_D$, $\mathsf{addfile}_{F \to D \to D}$.

New axioms:

13. TOTAL2(addfile) (addfile is total).
14. INJ2(addfile) (addfile is injective).
15. $\forall\, f : F,\, d : D \,.\, \mathsf{addfile}\ f\ d \neq \mathsf{emptydir}$ (disjointness).
16. $\forall\, p : \{D\} \,.\, p\ \mathsf{emptydir} \ \wedge\ \forall\, f : F,\, d : D \,.\, p\ d \Rightarrow p\ (\mathsf{addfile}\ f\ d) \Rightarrow \forall\, d : D \,.\, p\ d$ (structural induction).

Inductive type verification. The constructors for D are $K_D = \{\mathsf{emptydir}_D,\ (\mathsf{addfile}_{F \to D \to D},\ U_{\{F\}})\}$. Here emptydir is a nullary constructor (first kind) and $(\mathsf{addfile}, U_{\{F\}})$ is a fourth-kind constructor: its type is $F \to D \to D$ (in Curryed form), the quasitype $U_{\{F\}}$ indicates the first argument ranges over all of F, and the presence of D in the domain makes addfile recursive.

Checking the obligations (Chapter 10, Section 10.2 of [1]):

(a) $U_{\{F\}} \neq \emptyset_{\{F\}}$: holds since F is a base type and every base type domain is nonempty (frame condition F2, Definition 5.1 of [1]).

(b) $\mathsf{addfile}{\downarrow}$ as a total function: TOTAL2(addfile) (Axiom 13).

(c) TOTAL-ON for addfile: follows from Axiom 13.

(d) INJ2(addfile): Axiom 14. This ensures distinct file–directory pairs produce distinct directories.

(e) $\mathsf{emptydir} \notin \mathsf{ran}(\mathsf{addfile})$: follows from Axiom 15 (disjointness).

(f) Disjoint ranges: trivially satisfied (only one nonnullary constructor).

(g) Structural induction: Axiom 16.

Hence (D, K_D) is a recursive inductive type in $\mathsf{DIRECTORY}$. By the Inductive Types Theorem (Chapter 10 of [1]), in every standard model M of $\mathsf{DIRECTORY}$, D_D^M consists of exactly $I^M(\mathsf{emptydir})$ and all values of the form $I^M(\mathsf{addfile})(d_f, \sigma)$ for $d_f \in D_F^M$ and $\sigma \in D_D^M$. That is, D_D^M is isomorphic to the set of finite lists of elements of D_F^M.

Comparison with LIST-TYPE. The following table shows the structural parallel between $\mathsf{DIRECTORY}$ and a list type:

LIST	**DIRECTORY**	**Role**
L	D	inductive type
E (element)	F (file)	element type
nil_L	$\mathsf{emptydir}_D$	nullary constructor
$\mathsf{cons}_{E \to L \to L}$	$\mathsf{addfile}_{F \to D \to D}$	recursive constructor
TOTAL2(cons)	TOTAL2($\mathsf{addfile}$)	totality
$\mathsf{cons}\ e\ l \neq \mathsf{nil}$	$\mathsf{addfile}\ f\ d \neq \mathsf{emptydir}$	disjointness
INJ2(cons)	INJ2($\mathsf{addfile}$)	injectivity

6 Directory Development

We now define derived operations on directories and prove key properties. Following the lesson from Development Project 1 (see Section 9), all operations are *defined* in this development using definite descriptions, not introduced as theory extension constants.

Development Definition 6.1 (Directory Development).

Name: DIR-DEV.

Bottom theory: DIRECTORY.

Definitions and theorems:

Def 1 (contains). $\mathsf{contains}_{N \to D \to o} = \mathrm{I}\, g : N \to D \to o \,.\, \forall\, n : N \,.\, g\; n\; \mathsf{emptydir} = F_o \;\wedge\; \forall\, n : N,\, f : F,\, d : D \,.\, g\; n\; (\mathsf{addfile}\; f\; d) = (\mathsf{fname}\; f = n) \vee g\; n\; d$

Remark. The predicate $\mathsf{contains}\; n\; d$ tests whether a file with name n exists in directory d. It is defined by structural recursion on d: the empty directory contains nothing, and $\mathsf{addfile}\; f\; d$ contains name n if either f has name n or d already contains n.

Thm 1. $\mathsf{TOTAL2}(\mathsf{contains})$ (contains is total).

Thm 2. $\forall\, n : N \,.\, \mathsf{contains}\; n\; \mathsf{emptydir} = F_o$ (empty directory contains nothing).

Def 2 (size). $\mathsf{size}_{D \to \mathbb{N}} = \mathrm{I}\, g : D \to \mathbb{N} \,.\, g\; \mathsf{emptydir} = 0 \;\wedge\; \forall\, f : F,\, d : D \,.\, g\; (\mathsf{addfile}\; f\; d) = g\; d + 1$

Remark. The function $\mathsf{size}\; d$ returns the number of file entries in directory d.

Thm 3. $\mathsf{TOTAL}(\mathsf{size})$ (size is total).

Thm 4. $\mathsf{size}\; \mathsf{emptydir} = 0$ (empty directory has size zero).

Thm 5. $\forall\, f : F,\, d : D \,.\, \mathsf{size}\; (\mathsf{addfile}\; f\; d) = \mathsf{size}\; d + 1$ (addfile increments size).

Thm 6. $\forall\, d : D \,.\, \mathsf{size}\; d \geq 0$ (size is non-negative).

Definitions are validated and theorems are proved in Appendix A. Note that contains and size are defined using definite descriptions (I-terms): each definition asserts the existence of a unique function satisfying the given recursion equations. The validations prove that these definite descriptions are defined (i.e., the I-terms denote values).

7 Theory Morphism: Files to Directories

We exhibit a theory morphism from FILE to DIRECTORY to demonstrate that individual files embed into the directory structure. Intuitively, a single file f corresponds to the one-element directory addfile f emptydir.

Since DIRECTORY is obtained from FILE by a chain of extensions (FILE $\leq$ SECURED-FILE $\leq$ DIRECTORY), the constants mkfile, fname, and fcontent and the base types N, C, F already belong to the language of DIRECTORY. We define a *non-trivial* morphism that reinterprets the file type F as the directory type D.

To define the accessor translations, we use the fact that INJ2(addfile) (Axiom 14) allows us to extract the unique file from a single-entry directory. Define:

$$\mathsf{extract}_{D\to F} \;=\; \lambda\, d : D \,.\, \mathrm{I}\, f : F \,.\, d = \mathsf{addfile}\ f\ \mathsf{emptydir}$$

This definite description is defined on the image of the embedding (single-entry directories) by Axiom 14.

Theory Translation Definition 7.1 (Files to Directories)**.**

Name: FILE-to-DIR.

Source theory: FILE.

Target theory: DIRECTORY.

Base type mapping:

$N \;\mapsto\; N$ (names preserved)

$C \;\mapsto\; C$ (contents preserved)

$F \;\mapsto\; D$

Constant mapping:

$\mathsf{mkfile}_{N\to C\to F} \;\mapsto\; \lambda\, n : N \,.\, \lambda\, c : C \,.\, \mathsf{addfile}\ (\mathsf{mkfile}\ n\ c)\ \mathsf{emptydir}$

$\mathsf{fname}_{F\to N} \;\mapsto\; \lambda\, d : D \,.\, \mathsf{fname}\ (\mathsf{extract}\ d)$

$\mathsf{fcontent}_{F\to C} \;\mapsto\; \lambda\, d : D \,.\, \mathsf{fcontent}\ (\mathsf{extract}\ d)$

Verification. We must show that the image of each FILE axiom under this morphism is valid in DIRECTORY. The key observation is that on single-entry directories (those of the form addfile f emptydir), extract (addfile f emptydir) $= f$ by the uniqueness of the definite description (using INJ2(addfile), Axiom 14).

1. **TOTAL2(mkfile) maps to totality of the translation**: Since TOTAL2(addfile) (Axiom 13) and TOTAL2(mkfile) (Axiom 1), $\lambda\, n : N \,.\, \lambda\, c : C \,.$ addfile (mkfile n c) emptydir is total.

2. **INJ2(mkfile) maps to injectivity**: If addfile (mkfile n_1 c_1) emptydir $=$ addfile (mkfile n_2 c_2) emptydir, then mkfile n_1 c_1 $=$ mkfile n_2 c_2 by INJ2(addfile) (Axiom 14), and $n_1 = n_2$, $c_1 = c_2$ by INJ2(mkfile) (Axiom 2).

3. **Accessor axioms (Axioms 3–4)**: For a single-entry directory $d = \mathsf{addfile}\ (\mathsf{mkfile}\ n\ c)\ \mathsf{emptydir}$: $\mathsf{fname}\ (\mathsf{extract}\ d) = \mathsf{fname}\ (\mathsf{mkfile}\ n\ c) = n$ (Axiom 3), and similarly $\mathsf{fcontent}\ (\mathsf{extract}\ d) = c$ (Axiom 4).

4. **Totality of accessors (Axioms 5–6)**: Follows from the totality of fname, fcontent, and the definedness of extract on single-entry directories.

5. **Surjectivity (Axiom 7)**: For any single-entry directory d, $\mathsf{extract}\ d = f$ for some $f : F$, and the translated mkfile applied to fname f and fcontent f reconstructs d by Axiom 7 of FILE.

By the Morphism Theorem (Theorem 14.16 of [1]), all theorems of FILE transport to DIRECTORY under this morphism. This confirms that single files embed faithfully into the directory structure.

8 Examples

To illustrate the theories and development concretely, we consider a small symbolic instance consisting of two files and a directory containing them. Let $n_1, n_2, n_3 : N$ and $c_1, c_2 : C$, with $n_1 \neq n_2$, $n_3 \neq n_1$, and $n_3 \neq n_2$.

Example 8.1 (Secured files). Define

$$f_1 \;=\; \mathsf{protect}\ (\mathsf{mkfile}\ n_1\ c_1)\ \mathsf{rd}, \qquad f_2 \;=\; \mathsf{protect}\ (\mathsf{mkfile}\ n_2\ c_2)\ \mathsf{wr}.$$

By the axioms of FILE and SECURED-FILE:

$$\mathsf{fname}\ f_1 = n_1, \qquad \mathsf{fcontent}\ f_1 = c_1, \qquad \mathsf{perm}\ f_1 = \mathsf{rd},$$

and similarly

$$\mathsf{fname}\ f_2 = n_2, \qquad \mathsf{fcontent}\ f_2 = c_2, \qquad \mathsf{perm}\ f_2 = \mathsf{wr}.$$

Thus f_1 and f_2 are files with the expected names, contents, and permission levels.

File	**Name / Content**	**Permission**
f_1	(n_1, c_1)	rd
f_2	(n_2, c_2)	wr

Example 8.2 (A directory with two entries). Define

$$d \;=\; \mathsf{addfile}\ f_1\ (\mathsf{addfile}\ f_2\ \mathsf{emptydir}).$$

This is a flat directory with two entries. By unfolding the definition of contains:

$$\mathsf{contains}\ n_1\ d = T_o, \qquad \mathsf{contains}\ n_2\ d = T_o, \qquad \mathsf{contains}\ n_3\ d = F_o.$$

Indeed, d contains files with names n_1 and n_2, but no file with name n_3. By the recursive definition of size:

$$\mathsf{size}\ d = \mathsf{size}\ (\mathsf{addfile}\ f_1\ (\mathsf{addfile}\ f_2\ \mathsf{emptydir})) = \mathsf{size}\ (\mathsf{addfile}\ f_2\ \mathsf{emptydir}){+}1 = (\mathsf{size}\ \mathsf{emptydir}{+}1){+}1 = 2.$$

So the directory d has exactly two file entries.

Directory d
f_1
f_2

$$\mathsf{contains}\ n_1\ d = T_o, \qquad \mathsf{contains}\ n_2\ d = T_o, \qquad \mathsf{contains}\ n_3\ d = F_o, \qquad \mathsf{size}\ d = 2.$$

Example 8.3 (Embedding a file into a directory). The theory morphism FILE-to-DIR maps a file to a one-entry directory. In particular, the image of f_1 is

$$d_{f_1} \;=\; \mathsf{addfile}\ f_1\ \mathsf{emptydir}.$$

Likewise,

$$d_{f_2} \;=\; \mathsf{addfile}\ f_2\ \mathsf{emptydir}.$$

These are single-entry directories corresponding to the files f_1 and f_2. Intuitively, the morphism identifies an individual file with the simplest directory that contains exactly that file.

$$
\begin{aligned}
f_1 &\longmapsto \mathsf{addfile}\ f_1\ \mathsf{emptydir} \\
f_2 &\longmapsto \mathsf{addfile}\ f_2\ \mathsf{emptydir}
\end{aligned}
$$

These examples illustrate the intended behaviour of the six modules: PERM supplies the permission levels, FILE constructs files, SECURED-FILE assigns permissions to them, DIRECTORY builds flat directories from files, DIR-DEV defines directory operations, and FILE-to-DIR embeds files into the directory structure.

9 Conclusion

We have formalized a simple file system as a development graph in Alonzo consisting of six modules:

1. PERM: an enumeration inductive type specifying four permission levels.
2. FILE: a theory definition specifying files as name–content pairs with a constructor and accessors.
3. SECURED-FILE: a theory extension combining files with permissions via protect and perm.
4. DIRECTORY: a theory extension introducing directories as a recursive inductive type over secured files.
5. DIR-DEV: a development definition with two defined operations (contains, size) and six theorems, including totality proofs and algebraic properties.
6. FILE-to-DIR: a theory morphism showing that files embed into directories, validated by the Morphism Theorem (Theorem 14.16 of [1]).

The development forms a genuine graph (not a linear chain): PERM and FILE are independent base theories that merge into SECURED-FILE, which is then extended to DIRECTORY.

Lessons applied from Development Project 1. This project incorporates the following corrections from the feedback on Development Project 1 (80/100):

- All derived operations (contains, size) are defined in a development definition using definite descriptions, not introduced as theory extension constants (-16 in Project 1 for this error).
- Every definition is validated: the appendix proves that the RHS of each Def is defined, showing both existence and uniqueness (-4 in Project 1).
- Every function has a TOTAL theorem.
- All formulas have explicit type annotations on quantifiers (-2 each in Project 1).

A Proofs and Validations for DIR-DEV

A.1 Validation of Def 1 (contains)

We must show that the definite description $\mathrm{I}\, g : N \to D \to o \,.\, \ldots$ denotes a value, i.e., there exists a unique function $g : N \to D \to o$ satisfying:

$$\forall\, n : N \,.\, g\ n\ \mathsf{emptydir} = F_o \tag{R1}$$

$$\forall\, n : N,\ f : F,\ d : D \,.\, g\ n\ (\mathsf{addfile}\ f\ d) = (\mathsf{fname}\ f = n) \vee g\ n\ d \tag{R2}$$

Existence. Define g by: for fixed $n : N$, define $g\ n$ by structural recursion on the second argument $d : D$. By the structural induction principle (Axiom 16 of DIRECTORY), every $d : D$ is either emptydir or addfile $f\ d'$ for unique $f : F$ and $d' : D$. Define:

$$g\ n\ d\ =\ \begin{cases} F_o & \text{if } d = \mathsf{emptydir}, \\ (\mathsf{fname}\ f = n) \vee g\ n\ d' & \text{if } d = \mathsf{addfile}\ f\ d'. \end{cases}$$

This is well-defined by structural recursion (each recursive call is on a structurally smaller second argument). The expression $(\mathsf{fname}\ f = n) \vee g\ n\ d'$ is defined since fname is total (Axiom 5 of FILE), equality is always defined, and $\vee$ is total. Hence g satisfies (R1) and (R2).

Uniqueness. If g and h both satisfy (R1) and (R2), then for any fixed $n : N$, by structural induction on d:

- **Base:** $g\ n\ \mathsf{emptydir} = F_o = h\ n\ \mathsf{emptydir}$.
- **Step:** If $g\ n\ d = h\ n\ d$, then $g\ n\ (\mathsf{addfile}\ f\ d) = (\mathsf{fname}\ f = n) \vee g\ n\ d = (\mathsf{fname}\ f = n) \vee h\ n\ d = h\ n\ (\mathsf{addfile}\ f\ d)$.

Hence $g = h$. □

A.2 Thm 1: TOTAL2(contains)

Proof. We show $\mathsf{contains}\ n\ d{\downarrow}$ for all $n : N$, $d : D$ by structural induction on d.

Base case: $\mathsf{contains}\ n\ \mathsf{emptydir} = F_o{\downarrow}$.

Inductive step: Assume $\mathsf{contains}\ n\ d{\downarrow}$. Then $\mathsf{contains}\ n\ (\mathsf{addfile}\ f\ d) = (\mathsf{fname}\ f = n) \vee \mathsf{contains}\ n\ d$. Since $\mathsf{fname}\ f{\downarrow}$ (Axiom 5), equality is defined, $\mathsf{contains}\ n\ d{\downarrow}$ (IH), and $\vee$ is total, the result is defined.

By structural induction (Axiom 16), TOTAL2(contains). □

A.3 Thm 2: $\forall\, n : N \,.\, \mathsf{contains}\ n\ \mathsf{emptydir} = F_o$

Immediate from the definition of contains (equation R1). □

A.4 Validation of Def 2 (size)

We must show that $\mathrm{I}\, g : D \to \mathbb{N} \,.\, \ldots$ denotes a value.

Existence. Define g by structural recursion on d: $g\ \mathsf{emptydir} = 0$ and $g\ (\mathsf{addfile}\ f\ d) = g\ d + 1$. This is well-defined since the recursive call is on the structurally smaller argument d, and $+1$ is total on $\mathbb{N}$.

Uniqueness. By structural induction: if g and h both satisfy the recursion, then $g\ \mathsf{emptydir} = 0 = h\ \mathsf{emptydir}$, and if $g\ d = h\ d$ then $g\ (\mathsf{addfile}\ f\ d) = g\ d + 1 = h\ d + 1 = h\ (\mathsf{addfile}\ f\ d)$. Hence $g = h$. □

A.5 Thm 3: $\mathsf{TOTAL}(\mathsf{size})$

Proof. By structural induction on d.

Base: $\mathsf{size}\ \mathsf{emptydir} = 0{\downarrow}$.

Step: Assume $\mathsf{size}\ d{\downarrow}$. Then $\mathsf{size}\ (\mathsf{addfile}\ f\ d) = \mathsf{size}\ d + 1$. Since $\mathsf{size}\ d{\downarrow}$ (IH) and $+1$ is total on $\mathbb{N}$, the result is defined. □

A.6 Thm 4: $\mathsf{size}\ \mathsf{emptydir} = 0$

Immediate from the definition of size. □

A.7 Thm 5: $\forall\, f : F,\, d : D\ .\ \mathsf{size}\ (\mathsf{addfile}\ f\ d) = \mathsf{size}\ d + 1$

Immediate from the definition of size. □

A.8 Thm 6: $\forall\, d : D\ .\ \mathsf{size}\ d \geq 0$

Proof. By structural induction on d.

Base: $\mathsf{size}\ \mathsf{emptydir} = 0 \geq 0$.

Step: Assume $\mathsf{size}\ d \geq 0$. Then $\mathsf{size}\ (\mathsf{addfile}\ f\ d) = \mathsf{size}\ d + 1 \geq 0 + 1 = 1 \geq 0$. □

References

[1] W. M. Farmer, *Simple Type Theory: A Practical Logic for Expressing and Reasoning About Mathematical Ideas*, 2nd ed., Birkhäuser, 2025.

[2] W. M. Farmer and D. Y. Zvigelsky, "Monoid Theory in Alonzo: A Little Theories Formalization in Simple Type Theory," *Journal of Applied Logics*, vol. 12, no. 7, 2025.